



[Blog] How does DSP work? Part 1: Samples and buffers

☐ Blog
Entries

1
post

1
poster

draft



Reply

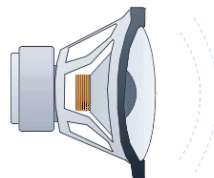


This first part starts before code.

We will begin at the end of the chain: the speaker.

Audio coding starts with understanding one simple thing:

A speaker makes sound by vibrating.



Top: Sound over time (waveform).

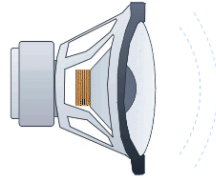
Bottom: Speaker cone follows that motion.

The line at the top is a waveform.

For now, think of it as the shape the speaker is following. When the line moves up, the speaker moves one way. When the line moves down, it moves the other way.

That movement pushes air, and we hear it as sound.

The first graphic is simplified so the motion is easy to see. Real audio is faster and messier:



Top: Vocal sample audio
Bottom: Speaker

The shape is more detailed now, but the idea is the same.

The speaker is still following a waveform.

So if we want to create or change a sound, we need some way to create or change that shape.

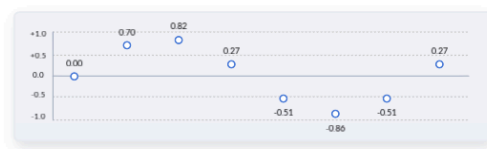
The shape as numbers

In digital audio, the shape is stored as numbers.

A tiny piece of a waveform might look like this:

```
[0.00, 0.70, 0.82, 0.27, -0.51, -0.86, -0.51, 0.27]
```

Plot those numbers from left to right and the waveform appears:



Each number is one **sample**.

A sample is just one height at one moment in time.

The order matters because the order is time:



The highlighted table cell and the highlighted point are the same moment.

The table is the data. The drawing is the same data made easier to see.

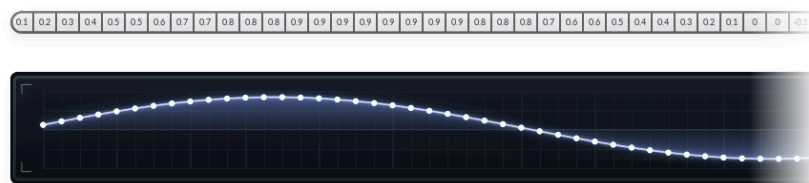
That is the first important idea:

the waveform is an array of sample values.

Lots of samples

The example above is tiny on purpose.

Real audio has far more samples than this. At 48 kHz, one second of mono audio contains 48,000 sample values.



"imagine this goes on for 48k samples...
That's one second of audio"

The simple drawing was not a different kind of thing. It was just a readable version of the same idea.

Real audio is the same thing, packed much more tightly in time.

Chunks

HISE does not work through the entire sound in one go.

It takes a short chunk of the waveform, processes it, then moves on to the next chunk.

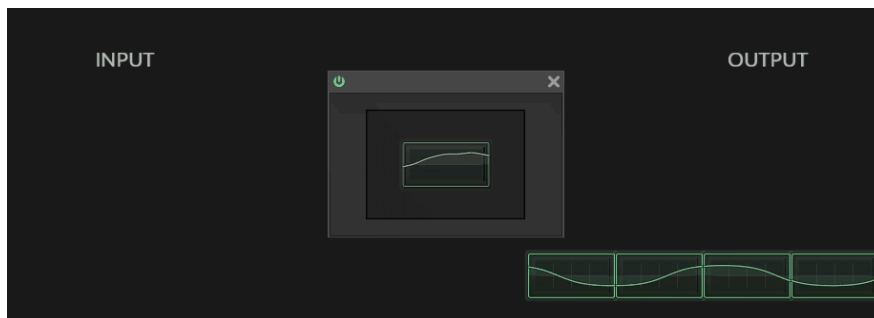


Those chunks are called **buffers**.

The chunk edges are not part of the sound. They do not need to line up with peaks, zero crossings, notes, or waveform cycles.

They are just the places where the engine cuts the stream into pieces it can process.

Now we can draw the basic movement through an effect:



A chunk comes in.

The effect changes the numbers inside it.

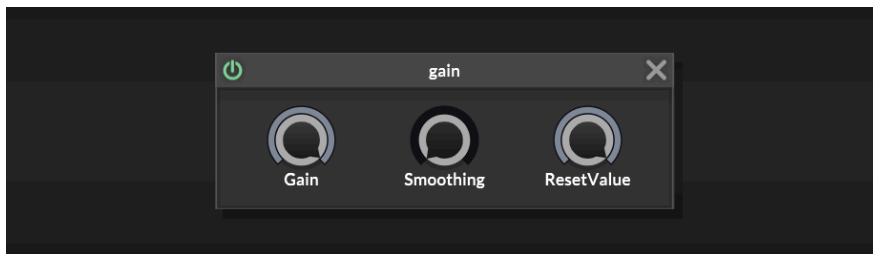
The chunk goes out.

Then the next chunk comes in.

This is the basic shape of a normal audio effect.

A simple example: volume

Volume is a good first example because it is easy to see.



Volume is the height of the waveform.



To make the sound quieter, make the sample values smaller.

For example, multiply every sample by 0.5:

Before



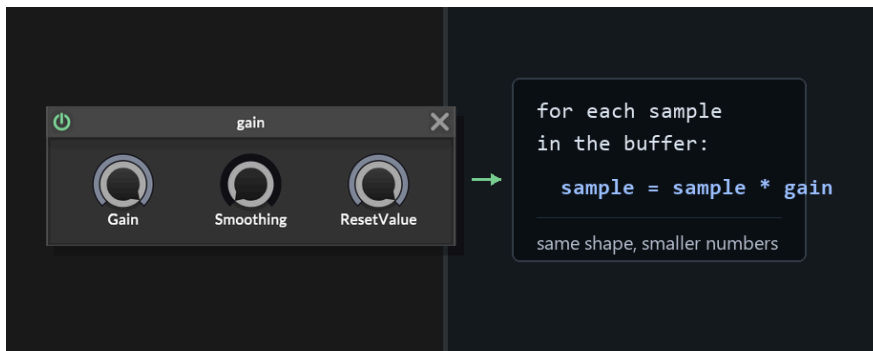
After x0.5



The shape is the same.

The height is half as large.

Inside the chunk, the operation is repeated for each sample:



In code form, the idea is:

```
for each sample in the buffer
{
    sample = sample * gain;
}
```

If **gain** is **1.0**, the sample values stay the same.

If **gain** is **0.5**, the waveform is half as tall.

If **gain** is **0.0**, every sample becomes zero, which is silence.

A real HISE gain node has controls and smoothing around it, but the core audio operation is still this simple multiplication.

The basic shape of DSP

Now we can say the fuller version.

For a normal audio effect, DSP usually looks like this:

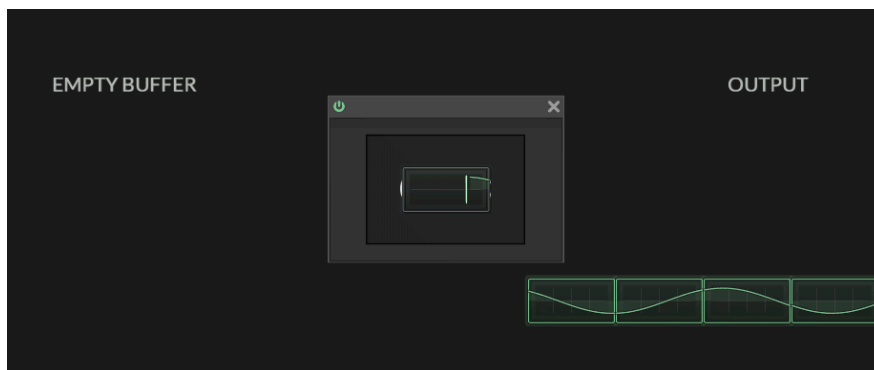
take a buffer of sample values, change those values, and pass the buffer onward.

Gain multiplies the samples.

Distortion bends or clips the samples.

Delay stores samples and plays them back later.

An oscillator is a slightly different case. It can create new sample values instead of changing incoming ones:



So the table, the waveform, the buffer, and the code are not separate ideas.

They are different views of the same thing:

sample values changing over time.

Next: how repeated values become pitch, tone, and spectrum.